

DEAKIN UNIVERSITY

SIT782 – Capstone Team Project (B)

Technical Contribution Report

**Automated Video Scraping, Duplicate Download Prevention
and Frame Dataset Generation**

Student Name	Sarvesh Venkatesh Vidya
Student ID	S225162255
Unit	SIT782 – Capstone Team Project (B)
Project	Redback Operations – E-waste Robotic Battery Extraction (Team 7)
Sub-team	Computer Vision
Contribution Area	Web scripting / automated dataset gathering
Primary Script	scripts/imgscraper.py
Test Video	iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4

Technical evidence and implementation results

1. Contribution Overview

As part of the Computer Vision work for the Redback Operations e-waste robotic battery extraction project, I contributed to the automated collection and preparation of visual data from mobile-phone battery repair and replacement videos. The existing image-scraping pipeline was designed to download videos from a YouTube playlist, extract useful frames, filter blurry frames, and apply segmentation before the resulting images were used as computer-vision dataset material.

A practical issue identified during repeated dataset collection was that the scraper could encounter videos that had already been downloaded. Re-downloading the same video would consume additional network bandwidth and storage and would also create unnecessary downstream processing. I therefore implemented and tested a duplicate-download check in the video acquisition stage of `imgscraper.py`.

The contribution was validated using an iPhone 14 battery replacement video. The first execution downloaded the video successfully. When the scraper was executed again, it detected the existing MP4 file and skipped the download while continuing with the existing local file for processing.

1.1 Contribution Objectives

- Prevent repeated downloading of videos that already exist in the local scraped-video directory.
- Allow an existing video to continue through the frame extraction and segmentation pipeline.
- Reduce unnecessary network usage, duplicate storage and repeated acquisition work.
- Verify the behaviour using an actual battery-replacement video rather than only testing the code path in isolation.
- Confirm that the duplicate-download check does not prevent the rest of the dataset-generation workflow from operating.

2. Existing Data Collection Workflow

The scraper forms one part of the computer-vision data preparation workflow. At a high level, the workflow takes a battery-related video, stores the source video locally, extracts frames at scene changes, removes blurry frames, and sends suitable frames to the segmentation stage.

1. Identify/download a relevant mobile-phone battery repair or replacement video.
2. Store the source MP4 in the local dataset/scraped directory.
3. Use FFmpeg to extract representative frames from the video.
4. Evaluate frames for blur and skip frames that do not meet the quality requirement.
5. Run segmentation on retained frames.
6. Save the resulting masked PNG images for later computer-vision dataset preparation.

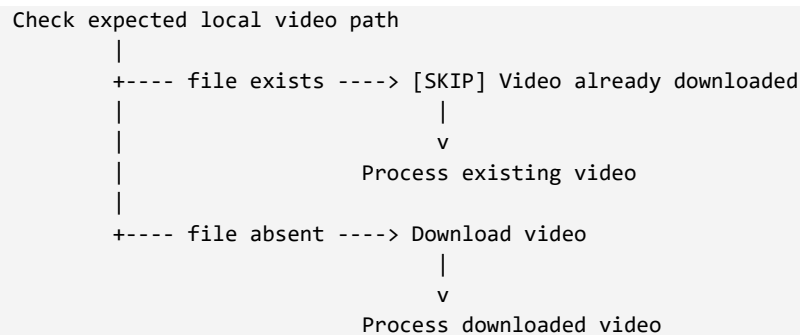
2.1 Problem with Repeated Execution

Dataset scripts are commonly executed multiple times while the collection process is being developed or expanded. Without a file-existence check, an already downloaded video may be downloaded again even though an identical local copy is available. This is inefficient and can become more significant as the playlist grows.

3. Duplicate-Download Prevention Implementation

The scraper was updated so that the video-download stage checks whether the expected local MP4 file is already present. If the file exists, the download operation is skipped and the existing file path is retained for subsequent processing.

3.1 Intended Control Flow



This approach separates acquisition from processing: the scraper does not need to download the source video again merely because the processing script has been run again. The existing local asset can be reused.

3.2 Verification Output

The first execution successfully downloaded the test video. The terminal output also showed the resulting local video path. A second execution then reported that the video was already downloaded and skipped the download operation.

4. Evidence of Duplicate-Download Prevention

```
(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> python imgscraper.py
[INFO] Downloading video: iPhone 14 Battery Dead? Replace it in No Time!
[INFO] Video resolution: 2160p
[INFO] Downloaded: ../dataset/scraped\iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
[TEST] Video path: ../dataset/scraped\iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> dir ../dataset/scraped\*.mp4

Directory: C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\dataset\scraped

Mode                LastWriteTime         Length Name
----                -
-a----             10-08-2026      20:04      93264369 iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
-a----             10-08-2026      19:18           0 test_video.mp4

(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> python imgscraper.py
[SKIP] Video already downloaded: iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
[TEST] Video path: ../dataset/scraped\iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
```

Figure 1. First download followed by a second execution. The second execution reports "[SKIP] Video already downloaded" for the existing iPhone 14 battery-replacement video.

The directory listing in Figure 1 also shows the existing MP4 file in the scraped dataset directory. The subsequent execution recognises that file instead of creating another copy.

5. Processing of the Existing Video

The duplicate check was designed not to terminate the workflow. After the existing video was detected, the script continued by processing that local file. This is important because the purpose of the check is to avoid redundant acquisition, not to skip useful downstream dataset processing.

5.1 Frame Extraction and Segmentation Evidence

```
(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> python imgscraper.py
[TEST] Processing existing video: ../dataset/scraped\iPhone_14_Battery_Dead_Replace_it_in_No_Time.mp4
[INFO] Extracting frames...
[INFO] Extracted 29 frames
[INFO] Running segmentation...
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0001_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0002_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0003_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0004_masked.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0005.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0006.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0007.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0008.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0009.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0010_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0011_masked.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0012.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0013.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0014.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0015.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0016_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0017_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0018_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0019_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0020_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0021_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0022_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0023_masked.png
[SAVE] ../dataset/scraped\masked\iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0024_masked.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0025.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0026.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0027.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0028.png
[SKIP] Blurry frame: iPhone_14_Battery_Dead_Replace_it_in_No_Time_frame_0029.png
[TEST] Existing video processing complete.
```

Figure 2. Processing of the existing video: 29 frames were extracted, blurry frames were skipped, and retained frames were passed to the segmentation stage and saved as masked PNGs.

The test run extracted 29 frames from the existing iPhone 14 video. The log shows multiple blurred frames being skipped and retained frames being saved as masked PNG files. The run ended with the message indicating that processing of the existing video was complete.

6. Test Results

Metric	Result	Interpretation
Test source video	iPhone 14 battery replacement video	Used as a real pipeline test case
Downloaded source video	1 MP4 file	Existing local video was reused on the second run
Frames extracted	29	All extracted frames were counted in the local frames directory
Frames retained after blur filtering	15	Suitable frames proceeded to segmentation
Blurry frames skipped	14	Approximately 48.3% of extracted frames were filtered

Masked outputs	15 PNG files	Retained frames produced segmentation outputs
Raw frame storage	52,504,386 bytes (~52.5 MB)	Total size of 29 extracted PNG frames
Masked output storage	8,094,470 bytes (~8.1 MB)	Total size of 15 masked PNG outputs

Based on the measured file totals, the masked outputs occupy approximately 15.4% of the storage used by the extracted raw frames in this test. This corresponds to an approximate 84.6% reduction in storage relative to the 29 raw extracted frames, while the number of files is reduced from 29 to 15.

6.1 Output Count Verification

```
(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> dir ..\dataset\scraped\frames\*.png | Measure-Object

Count       : 29
Average     :
Sum         :
Maximum     :
Minimum     :
Property    :

(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> dir ..\dataset\scraped\masked\*.png | Measure-Object

Count       : 15
Average     :
Sum         :
Maximum     :
Minimum     :
Property    :

(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> Get-ChildItem ..\dataset\scraped\frames\*.png | Measure-Object -Property Length -Sum

Count       : 29
Average     :
Sum         : 52504386
Maximum     :
Minimum     :
Property    : Length

(.venv) PS C:\Users\sarve\redback-e-waste-battery-extraction-computer-vision\scripts> Get-ChildItem ..\dataset\scraped\masked\*.png | Measure-Object -Property Length -Sum

Count       : 15
Average     :
Sum         : 8094470
Maximum     :
Minimum     :
Property    : Length
```

Figure 3. PowerShell verification of the generated dataset: 29 raw extracted frames and 15 masked PNG outputs, together with their total file sizes.

7. FFmpeg Compatibility Adjustment

During testing of the frame-extraction stage with FFmpeg 9.0, the existing command used the older vsync option for variable-frame-rate handling. The option was updated from vsync='vfr' to fps_mode='vfr' to match the newer FFmpeg interface. After this adjustment, frame extraction completed successfully and produced the 29-frame test result shown in the evidence.

This was a compatibility adjustment rather than a change to the dataset-selection logic. It was necessary to ensure that the acquisition and extraction pipeline remained executable in the current development environment.

8. Technical Outcome and Contribution

- The scraper can now recognise a locally existing video and avoid an unnecessary second download.

- The existing video remains available to the downstream frame-extraction and segmentation stages.
- Repeated execution of the scraper is therefore more suitable for iterative dataset development.
- The test demonstrates both sides of the workflow: initial acquisition and subsequent reuse of the same local asset.
- The frame-generation stage was also validated on the reused video, producing 29 extracted frames and 15 masked outputs.
- The contribution reduces avoidable network and storage overhead at the video-acquisition stage and supports a more repeatable dataset-generation workflow.

8.1 Relevance to the Computer Vision Pipeline

Although the duplicate-download check is a relatively small part of the overall computer vision system, it supports the reliability of the data-generation pipeline. The later computer-vision stages depend on having a consistent collection of source videos and processed frames. Preventing duplicate acquisition helps maintain a cleaner dataset directory and makes repeated testing less wasteful.

9. Limitations and Future Improvements

- The current duplicate check is based on the expected local video file being present; a stronger dataset-management approach could additionally use video IDs or metadata to identify the source uniquely.
- A file that exists but is incomplete or corrupted would still require additional validation if the pipeline is used at a larger scale.
- The segmentation output still requires quality review because a generic segmentation prompt can produce partial or irrelevant masks.
- Future versions could maintain a manifest containing source video IDs, filenames, processing status and timestamps to make the dataset easier to audit.

10. Conclusion

The duplicate-download prevention task was implemented and verified as part of the automated video-scraping and dataset-generation workflow. The evidence demonstrates that the first execution downloaded the iPhone 14 battery-replacement video, while a subsequent execution detected the existing MP4 and skipped the download. Importantly, the existing video was then processed successfully, with 29 frames extracted, 14 blurry frames filtered out and 15 masked outputs generated.

The contribution therefore addresses both efficiency and repeatability: source videos do not need to be downloaded again when they are already available locally, while the downstream computer-vision processing can still be performed when required.

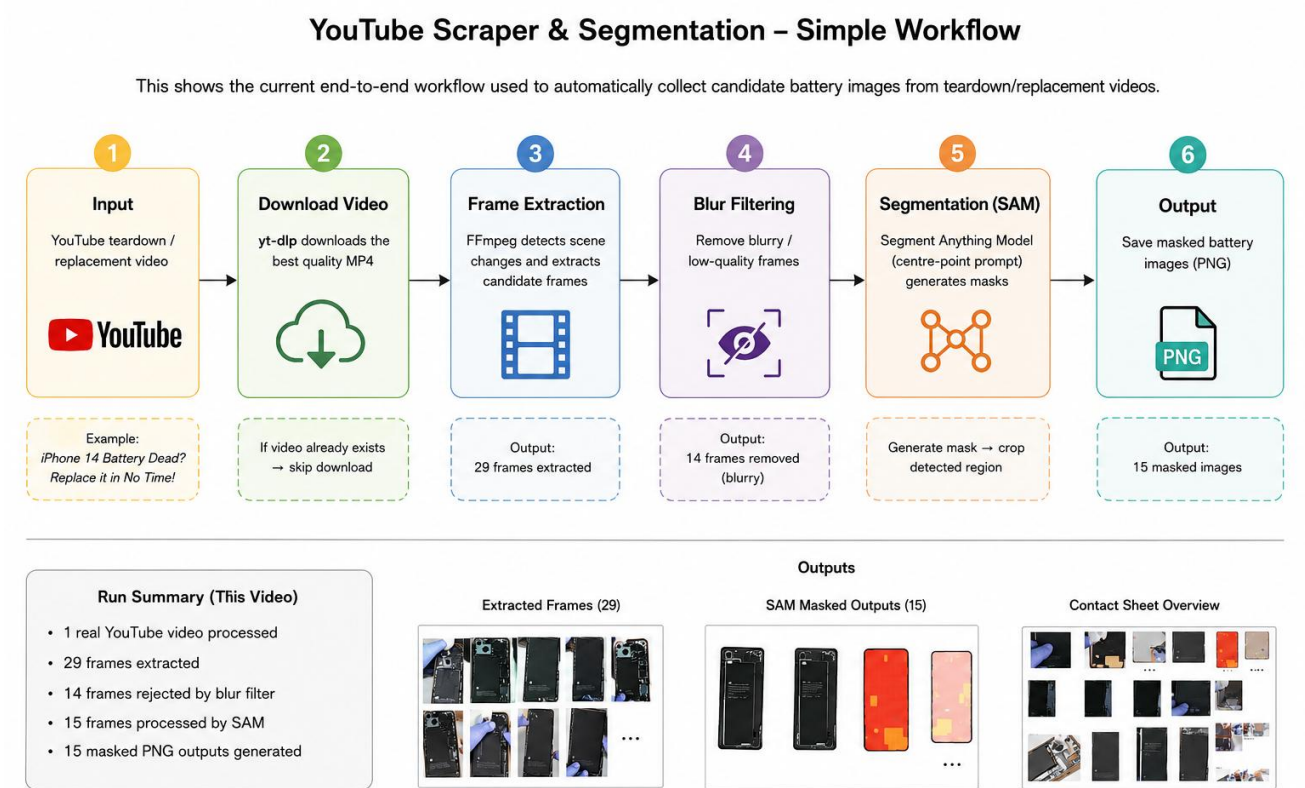
11. Evidence Summary

Evidence	What it demonstrates	Figure
Terminal download / skip log	Initial download and duplicate detection on second execution	Figure 1
Processing log	Existing video continues through frame extraction, blur filtering and segmentation	Figure 2

PowerShell count and size checks	29 raw frames and 15 masked outputs, with storage totals	Figure 3
----------------------------------	--	----------

Appendix A – End-to-End YouTube Scraper and Segmentation Workflow

The following workflow diagram provides a visual summary of the complete data-generation pipeline used in this contribution. It shows where the duplicate-download check sits within the wider process: after the YouTube video is identified, the scraper checks whether the video already exists before proceeding to frame extraction, blur filtering, segmentation, and output generation.





iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



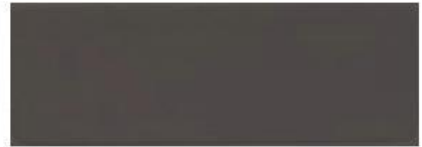
iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



iPhone_14_Battery_Dead_Replace_it_in_N



Figure B1. Contact sheet of masked outputs generated from the iPhone 14 battery-replacement video. The examples show variation in the detected region across frames.

The contact sheet also illustrates why the generated outputs require later quality review. Some frames contain clear battery regions, while other outputs can include partial regions or content affected by the centre-point segmentation prompt. This is a limitation of the current segmentation approach rather than the duplicate-download mechanism.

Appendix C – Visual Evidence Mapping

Visual Evidence	Purpose	Related Report Section
Figure A1 – Workflow diagram	Shows the complete scraper-to-segmentation workflow and where duplicate checking occurs.	Sections 2–3
Figure B1 – Masked contact sheet	Shows the actual segmentation outputs generated during the test run.	Sections 5–6
Figure 1 – Terminal download/skip evidence	Demonstrates initial download and duplicate detection on the second execution.	Section 4
Figure 2 – Processing log	Demonstrates continued processing of the existing video.	Section 5
Figure 3 – File count/size verification	Confirms 29 raw frames and 15 masked outputs and their storage totals.	Section 6